

C 역할 준비를 위한

2 주 집중 학습 계획

Transformer 내부 구조부터 KV Cache 최적화 아이디어까지

주간 학습 시간	2 주 총 학습 시간	월요일
24 시간	48 시간	휴식 / 일정 비움

2 주 목표

작은 모델에서 Transformer 레이어와 Q·K·V 를 찾고, `past_key_values` 와 KV Cache 가 생성·갱신되는 과정을 관찰한다. Forward hook 과 Hugging Face Cache 구조를 익힌 뒤, 대표 KV Cache 최적화 논문을 구현 관점에서 비교하여 로그 특화 압축 아이디어의 기반을 만든다.

왜 이 2 주를 먼저 해야 하나

C 역할의 실제 과제는 토큰-라인 매핑, 외부 마스크 기반 KV 삭제, `kvpress` 확장, LogPress 구현이다. 이 작업들은 Transformer 내부의 어느 텐서가 토큰 위치를 나타내는지 모르면 디버깅조차 어렵다. 따라서 첫 2 주는 연구 아이디어보다 “내부를 관찰하고 안전하게 수정하는 능력”을 확보하는 준비 단계다.

이 문서의 사용법

- 화~금은 하루 5 시간, 토·일은 하루 2 시간만 수행한다.
- 논문은 전문 정독보다 문제, 선택 기준, 압축 단위, 한계 중심으로 읽는다.
- 모든 실습에서 중간 Tensor shape 를 출력하고 실행 결과를 짧게 기록한다.
- 2 주 안에 LogPress 를 완성하지 않는다. 구조 이해와 간단한 압축 toy 구현이 목표다.
- 매일 “무엇을 했는가”뿐 아니라 “이 작업이 C 역할의 어느 문제를 해결하는가”를 한 문장으로 기록한다.

1. 2 주 동안 갖춰야 할 핵심 역량

핵심 기준

개념을 암기하는 것이 아니라 작은 모델에서 해당 구조를 직접 찾고, 출력하고, 수정한 증거를 남기는 것이 목표다.

필요 역량	왜 해야 하나	2 주 후 증거
Transformer 텐서 구조 읽기	KV 를 삭제하려면 [B, Hkv, T, Dh]에서 토큰 위치가 T 축이라는 사실을 정확히 알아야 한다. 축을 잘못 자르면 head 나 feature 를 삭제하는 치명적 오류가 난다.	Hidden state, Q, K, V, attention score 의 모든 축을 말로 설명한다.
MHA 직접 구현	직접 구현해야 QKV projection, head 분리, causal mask, softmax, V 가중합이 실제 코드에서 어떻게 연결되는지 이해할 수 있다. 이후 논문 코드의 변형 지점을 찾기 쉬워진다.	MHA 코드와 각 단계 shape 로그를 제출한다.
실제 모델 해부	toy 코드와 실제 Hugging Face 모델은 모듈 이름과 반환 형식이 다르다. C 는 남의 코드를 확장해야 하므로 block, attention, projection, MLP 위치를 스스로 찾아야 한다.	tiny-gpt2 와 Qwen 에서 대상 모듈 경로를 기록한다.
KV Cache 추적	압축은 Cache 가 언제 만들어지고 어디에 이어 붙는지 알아야 가능하다. Prefill 과 Decode 를 구분하지 못하면 압축 시점과 성능 측정을 설계할 수 없다.	$T \rightarrow T+1 \rightarrow T+2$ Cache 길이 변화를 표로 남긴다.
Forward hook 사용	kvpress 가 막히거나 내부 텐서를 관찰해야 할 때 hook 이 빠른 진입점이 된다. 특정 레이어의 입력·출력을 바꿔 최종 결과 변화까지 확인할 수 있어야 한다.	hook 전후 logits 차이와 handle.remove() 복원 결과를 기록한다.
Cache 클래스 이해	라이브러리 방식이 실패하면 Cache update 를 직접 확장하는 백업이 필요하다. C 역할의 PoC 리스크를 낮추는 핵심 안전장치다.	DynamicCache update 흐름과 최근 N 개 보존 실험을 보여준다.
논문을 구현 관점으로 읽기	아이디어를 만들려면 기존 방법이 무엇을 남기고 무엇을 잃는지 알아야 한다. 특히 attention 중심 방법이 로그 원인 증거를 놓치는 경우를 찾아야 차별점이 생긴다.	StreamingLLM, H2O, SnapKV, PyramidKV, FastKV 비교표를 완성한다.

공통 실습 모델과 도구

모델·라이브러리	기록할 항목
☐ sshleifer/tiny-gpt2: 구조 확인용 ☐ Qwen2.5-0.5B-Instruct: GQA 와 Cache 관찰용 ☐ PyTorch ☐ Hugging Face Transformers	☐ torch·transformers 버전 ☐ 모델 레이어 수·Q head·KV head ☐ 모든 중간 Tensor shape ☐ 정상 실행과 수정 실행의 logits 차이

2. 1 주차 - Transformer · Hook · Cache 기본기

1 주차 최종 상태

직접 만든 MHA 와 실제 작은 모델의 attention 구조를 연결하고, hook 으로 중간 출력을 관찰·수정하며, 토큰 생성에 따라 KV Cache 길이가 늘어나는 과정을 설명할 수 있어야 한다.

화요일 · 5 시간 Transformer 와 MHA

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
Attention Is All You Need 의 Figure 1, 3.1, 3.2, 3.2.1, 3.2.2 를 읽는다. Single/Multi-Head Attention 을 구현하고 causal mask 와 모든 중간 shape 를 출력한다.	KV 압축은 결국 K·V 텐서의 특정 토큰 위치를 선택하는 작업이다. MHA 계산 흐름을 모르면 어떤 텐서가 Cache 로 저장되는지, 어느 축을 삭제해야 하는지 판단할 수 없다.	mha.py 실행. Q·K·V 와 score shape 설명. Softmax 행 합과 causal mask 확인.

수요일 · 5 시간 실제 GPT 구조와 Forward hook

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
간단한 Transformer block 을 구현한다. tiny-gpt2 구조를 출력하고 block·attention·MLP 에 forward hook 을 등록한다. 출력 일부를 0 으로 바꿔 logits 를 비교한다.	프로젝트에서는 완성된 라이브러리 내부에 개입해야 한다. hook 은 소스 전체를 뜯어고치기 전에 어느 레이어에서 어떤 값이 흐르는지 확인하는 가장 빠른 디버깅 도구다.	transformer_block.py, forward_hook.py. hook 제거 후 원래 결과가 복원되는지 확인.

목요일 · 5 시간 past_key_values 와 수동 Decode

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
Qwen2.5-0.5B 구조를 분석하고 use_cache=True 로 past_key_values 를 확인한다. 레이어별 K·V shape 를 출력하고 generate() 없이 토큰 3 개를 수동 생성한다.	KV Cache 최적화의 대상이 실제로 어떻게 생기고 늘어나는지 보아야 한다. Prefill 과 Decode 를 직접 분리하면 압축을 언제 적용할지, 압축 비용을 어디서 측정할지 이해할 수 있다.	manual_decode.py. Prefill T → Decode T+1 → T+2 → T+3 표 작성.

금요일 · 5 시간 Hugging Face Cache 클래스

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
공식 Cache 설명과 cache_utils.py 를 읽는다. DynamicCache 의 update, get_seq_length, crop 흐름을 확인하고 최근 N 개만 남기는 간단한 Cache 실험을 한다.	파일 일정에서 kvpress 혹은 실패하면 Cache 직접 확장이 백업 경로다. update 흐름을 이해하면 외부 마스크를 KV 에 적용하는 PoC 로 넘어갈 수 있다.	recent_window_cache.py. 정상 Cache 와 제한 Cache 의 길이·logits 차이 기록.

토요일 · 2 시간 MQA 와 GQA

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
MQA 논문과 GQA 논문의 Abstract, Introduction, 핵심 구조 그림을 읽는다. MHA·GQA·MQA 의 KV head 수와 메모리 차이를 정리한다.	실제 Llama/Qwen 계열은 Query head 수와 KV head 수가 다를 수 있다. 이를 모르고 mask 를 만들면 head 차원을 잘못 처리할 수 있다.	MHA: $H_q = H_{k_v}$, GQA: $H_q > H_{k_v} > 1$, MQA: $H_{k_v} = 1$ 비교표.

일요일 · 2 시간 1 주차 통합·복습

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
Qwen 입력 → hook → Cache 생성 → 수동 Decode → 최근 N 개 보존 흐름을 한 파일에서 실행한다. 막힌 항목을 목록으로 만든다.	각 실습이 따로 돌아가는 것과 하나의 흐름으로 연결되는 것은 다르다. 통합 실행을 해야 C 역할에서 실제 디버깅 순서를 경험할 수 있다.	week1_summary.md, 실행 로그, 막힌 항목과 다음 주 수정 계획.

1 주차 필수 질문

- $Q \cdot K \cdot V$ 는 같은 입력에서 만들어지는데 왜 서로 다른 projection 을 쓰는가?
- 왜 일반적으로 Q 는 저장하지 않고 $K \cdot V$ 만 Cache 에 저장하는가?
- KV Cache 에서 sequence 축은 어느 축이며, 왜 그 축을 잘못 다루면 안 되는가?
- Prefill 과 Decode 는 무엇이 다르고, 압축 시점과 어떤 관계가 있는가?
- Forward hook 은 언제 호출되며, 출력 수정은 최종 logits 에 어떤 영향을 주는가?

3. 2 주차 - KV Cache 최적화 논문과 아이디어

2 주차 최종 상태

StreamingLLM, H2O, SnapKV, PyramidKV, FastKV의 선택 기준을 비교하고 간단한 toy 정책을 구현한다. 마지막에는 로그 RCA에 적용할 수 있는 압축 아이디어 후보를 최소 3개 작성한다.

화요일 · 5 시간 StreamingLLM

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
attention sink와 초기 토큰+최근 토큰 보존 원리를 정리한다. Recent-only와 Sink+Recent toy 정책을 구현해 보존 위치와 logits를 비교한다.	가장 단순한 위치 기반 baseline을 알아야 로그 특화 방법의 추가 이득을 설명할 수 있다. 또한 최근 구간만 남기면 중간 원인 증거를 잃을 수 있다는 문제를 확인할 수 있다.	streaming_policy.py. 두 정책의 보존 위치와 logits 차이 비교.

수요일 · 5 시간 H2O

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
Heavy Hitter 누적 attention 점수를 이해한다. toy attention에서 누적 score top-k + 최근 window를 구현한다.	H2O는 "많이 주목받은 토큰을 남긴다"는 대표적인 attention 기반 기준이다. 로그에서는 결과 ERROR가 많이 주목받고 실제 원인 INFO가 덜 주목받을 수 있으므로 한계를 발견하는 기준선이 된다.	h2o_toy.py. attention이 높음과 RCA 원인이 같지 않은 사례 2개 작성.

목요일 · 5 시간 SnapKV

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
observation window와 head 별 선택을 이해한다. 마지막 W개 query의 attention으로 head 별 top-k 위치를 선택한다.	프로젝트의 head-aware 아이디어와 가장 직접적으로 연결된다. 서로 다른 head가 다른 위치를 고르는 현상을 확인해야 head 별 압축률 차등 가설을 세울 수 있다.	snapkv_toy.py. 공통 top-k와 head 별 top-k의 보존 위치 차이 출력.

금요일 · 5 시간 PyramidKV·FastKV와 아이디어 설계

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
레이어별 Cache 예산과 초기 레이어 보존 논리를 비교한다. line/chunk-aware, head-aware, layer-aware 후보를 설계하고 각 후보의 반증 실험을 작성한다.	파일의 핵심 설계는 "판단은 한 번, 적용은 초기 레이어 이후"다. layer-aware 논문을 읽어야 이 선택의 근거와 실험 대조군을 만들 수 있다.	아이디어 3종과 각 아이디어의 가장 빠른 반증 실험.

토요일 · 2 시간 NVIDIA kvpress 구조

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
BasePress, ScorerPress, compress(), pipeline 호출 흐름을 추적하고 기존 press 2~3 개를 실행할 준비를 한다.	파일 일정에서 C 는 3~6 주차부터 kvpress 구조를 조사해야 한다. 2 주 안에 구조를 미리 봐두면 프로젝트 시작 후 학습 시간이 아니라 PoC 구현 시간으로 사용할 수 있다.	kvpress_flow.md, pipeline → press → compress → compressed KV 흐름도.

일요일 · 2 시간 2 주 전체 정리

무엇을 학습할까	왜 해야 하나	완료 기준 / 결과물
기존 방법 비교표를 작성하고 토큰-라인 매핑·외부 마스크 KV 삭제 PoC 로 이어질 학습 공백을 점검한다.	논문 요약만 남기면 다음 구현으로 연결되지 않는다. 어떤 코드와 지식이 부족한지 확인해야 3 주차 이후 일정 지연을 막을 수 있다.	final_notes.md, 기존 방법 비교표, 다음 구현 우선순위 3 개.

4. 읽을 논문과 “왜 읽는가”

논문 / 자료	무엇을 배울까	왜 지금 읽나
Attention Is All You Need	Scaled Dot-Product Attention, Multi-Head Attention, Transformer block	모든 KV Cache 최적화 방법이 전제로 삼는 $Q \cdot K \cdot V$ 생성과 attention 계산을 이해하기 위해서다.
Fast Transformer Decoding: One Write-Head is All You Need	MQA 와 KV head 공유	KV head 수가 Cache 크기와 메모리 대역폭에 어떤 영향을 주는지 이해하기 위해서다.
GQA	MHA 와 MQA 사이의 grouped KV 구조	Qwen/Llama 계열의 실제 Cache shape 를 올바르게 해석하기 위해서다.
StreamingLLM	attention sink + 최근 window	가장 단순한 위치 기반 보존법과 중간 증거 손실 문제를 baseline 으로 삼기 위해서다.
H2O	누적 attention 기반 Heavy Hitter 보존	attention 중요도와 RCA 원인 중요도가 다를 수 있다는 연구 가설을 만들기 위해서다.
SnapKV	observation window 와 head 별 top-k 선택	head-aware 로그 압축 아이디어를 구체적인 구현 단위로 연결하기 위해서다.
PyramidKV	레이어별 차등 Cache 예산	모든 레이어에 같은 압축률을 적용하지 않는 layer-aware 설계를 이해하기 위해서다.
FastKV	초기 레이어 전체 처리 후 선택적 축소	프로젝트의 “초기 레이어 이후 적용” 설계를 방어 하고 ablation 대조군을 만들기 위해서다.
Hugging Face cache_utils.py	DynamicCache, update, crop, layer 별 저장 구조	kvpress 실패 시 Cache 직접 확장이라는 백업 경로를 확보하기 위해서다.
NVIDIA kvpress	BasePress, ScorerPress, compress(), pipeline	파일 일정의 PoC 와 LogPress 구현이 실제로 들어갈 확장 지점을 찾기 위해서다.

논문 읽기 규칙

- 첫 번째 읽기: Abstract → Introduction → 핵심 Method 그림 → Main result → Limitation 순서로 본다.
- 각 논문마다 “무엇을 남기는가, 어떤 기준으로 고르는가, 언제 압축하는가, 무엇을 놓칠 수 있는가”만 먼저 정리한다.
- 수식을 전부 증명하지 않는다. 이번 2 주의 목적은 논문 아이디어를 toy 코드와 프로젝트 설계로 연결하는 것이다.
- 로그 RCA 관점에서 결과 ERROR, 이전 원인 INFO, 반복 템플릿, 중간 위치 증거가 어떻게 처리될지 반드시 적는다.

5. 2 주 종료 통과 기준

통과의 의미

아래 항목을 대부분 직접 실행할 수 있으면 C 역할의 본 일정에 진입할 준비가 된 것이다. 모든 논문을 완벽히 이해했는지는 통과 기준이 아니다.

구조·Cache 실습

- MHA 를 직접 구현하고 모든 중간 shape 를 설명한다.
- 작은 모델에서 attention·MLP·LayerNorm 을 찾는다.
- hook 으로 입력·출력을 저장하고 출력을 수정한다.
- past_key_values 에서 레이어별 K·V shape 를 출력한다.
- 수동 Decode 에서 Cache 길이가 1 씩 증가함을 보인다.

최적화·아이디어

- DynamicCache 의 update 흐름을 소스에서 찾는다.
- 최근 N 개를 남기는 간단한 Cache 정책을 실행한다.
- StreamingLLM·H2O·SnapKV toy 정책을 설명한다.
- PyramidKV·FastKV 의 layer-aware 관점을 설명한다.
- 로그 특화 압축 후보를 최소 3 개 작성한다.

완료 후 바로 이어질 다음 단계

다음 구현 흐름

토큰나이저 offsets_mapping → token_idx ↔ line_idx 매핑 → 특정 로그 라인을 token mask 로 변환 → K·V 의 sequence 축에 외부 마스크 적용 → 초기 레이어 이후에만 삭제하는 PoC 순서로 진행한다.

왜 이 순서인가

로그 한 줄을 정확히 토큰 위치로 바꾸지 못하면 어떤 KV 를 지웠는지 증명할 수 없다. 따라서 압축 점수보다 매핑 정확성과 단위 테스트가 먼저이며, 그 뒤에 kvpress 또는 Cache 확장으로 실제 삭제를 연결해야 한다.

6. 매일 남길 기록 형식

- 오늘 실행한 파일과 모델 버전
- 입력 shape → 중간 shape → 출력 shape
- 정상 실행과 수정 실행의 차이
- 오늘 작업이 C 역할의 어떤 문제를 해결하는지 한 문장
- 막힌 오류와 다음에 가장 먼저 확인할 가설 1 개